

- The system will help manage multiple **financial advisors** clients. (*FinancialAdvisor entity will store advisor details and Client entity stores client data*)
- Financial advisors must be able to **create, update, and remove clients**. (*CRUD and also confirms FinancialAdvisor is primary actor responsible for maintaining accuracy of clients data*)
- Each financial advisor **can have numerous clients**. (*One-to-many 1:M*) relationship between FinancialAdvisor and Client. AdvisorID in Client table enforces link
- **Financial advisors will be using the system during standard business hours from 9 to 5 on weekdays**. (*Operational*)
- Each client will have a **portfolio**. (*One-to-one 1:1*) relationship between Client and Portfolio. Foreign key (ClientID) enforces each portfolio belongs to unique client
- Client portfolios may **contain zero or more securities**. (*One-to-many 1:M*) relationship between Portfolio and Security. Foreign key in Security table links each security to its client's portfolio
- Financial advisors must be able to **create, update, and remove securities from client portfolios**. (*Reinforces Portfolio <---> Security relationship. As securities managed from client portfolios, system has to ensure every Security record is linked correctly to single Portfolio via PortfolioID foreign key*)
- Every **security** has a name, a category, a purchase date, a purchase price, and a **quantity**. (*Attributes defined for Security entity*)
- **The system must have 99% uptime**. (*Non-functional*)
- **The system must expose a React dashboard**. (*Front end*)
- **The system's backend must use the Spring framework for Java**. (*Back end*)
- **The system must store data in a relational database**. (*Only confirms choice of ERD*)
- **The system must be highly scalable**. (*Non-functional*)

